

数据库说明

1. 数据结构

代码块

```
1 topfd/  
2   ms1_json/spectrum*.js  
3   ms2_json/spectrum*.js  
4   toppic_prsm_cutoff/data_js/  
5     proteins.js  
6     prsms/prsm*.js  
7   toppic_proteoform_cutoff/data_js/  
8     proteins.js  
9     prsms/prsm*.js
```

入库内容为黄色标

数据内容	存在哪里
数据集名、slug、原始数据根路径、cutoff 元数据	表 datasets、cutoffs；路径在 datasets.source_path / cutoffs.data_path
蛋白/异构体列表与统计	表 proteins、proteoforms
PrSM 的标量 + 嵌套结构 (annotated、ms 头、ms.peaks 等)	表 prsms 及列 annotated_protein、ms_header、ms_peaks (JSONB)

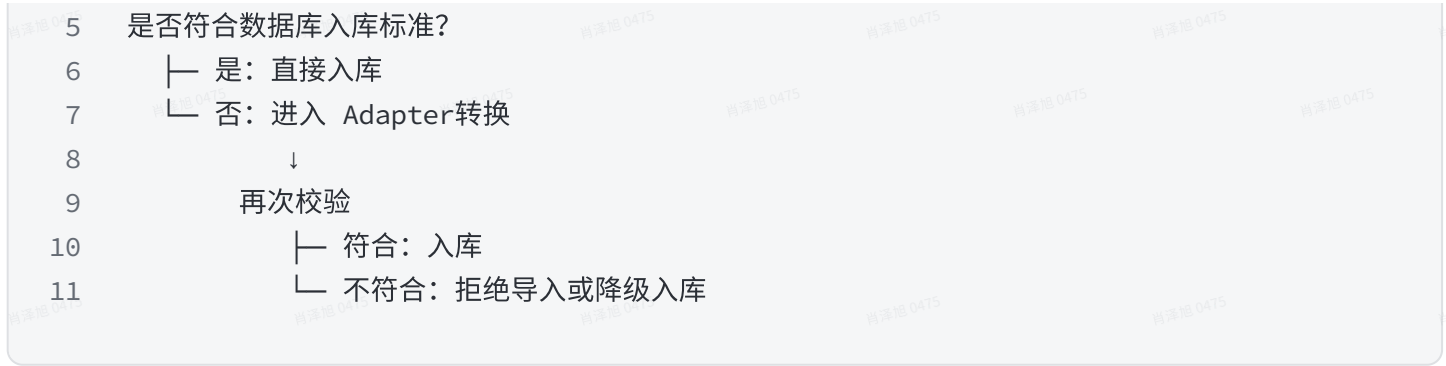
谱图数据不入库（数据库只记录谱图所在的位置），画 MS1/MS2 全谱时，后端用 `source_path` 去读对应 JS 文件（并可 LRU 缓存在进程内存，不是数据库）

2. 优化方向

建立一个统一适配层，由这一层将数据全部进行转化成我需要的数据。
核心思想：符合标准直接入库；不符合就转换成标准；转不了就拒绝或降级
流程：

代码块

```
1  外部数据  
2  ↓  
3  数据准入模块  
4  ↓
```



3. 数据准入模块

新增一个逻辑模块，暂名：Data Admission

它负责：

1. 识别数据来源格式。
2. 检查数据是否已经符合数据库入库标准。
3. 若不符合，调用对应 Adapter 转换。
4. 转换后再次校验。
5. 合格后写入 PostgreSQL。
6. 记录导入日志、错误、来源文件和能力信息。

3.1 内部流程



3.2 关键原则

- 数据库是标准。
- Adapter 负责适配外部格式。
- 前端只消费数据库/API 提供的统一结构。
- 不合格数据不能静默入库。
- 可降级入库的数据必须明确标记能力缺失。

4. 数据库入库标准

4.1 Dataset

4.1.1 现有字段

当前 `datasets` 表已有：

- `id`
- `slug`
- `name`
- `description`
- `source_path`
- `created_at`
- `updated_at`

4.1.2 必须满足

入库时必须保证：

- `slug` 全局唯一。
- `name` 非空。
- `source_path` 指向存在的数据根目录，或指向可被后端解析的数据位置。
- 如果该数据集需要画谱，`source_path` 必须能最终定位谱图文件。

4.1.3 未来新增字段

为了支持多格式导入，建议在 `datasets` 增加：

- `source_software`：来源软件，例如 `toppic_topfd`、`vendor_x`。
- `profile`：数据集类型，例如 `topdown_prsm`、`spectrum_only`。
- `raw_data_path`：原始数据归档路径。
- `standard_data_path`：标准化后文件根目录，可空。

- `capabilities`：JSONB，记录支持哪些功能。
- `extra_metadata`：JSONB，兜底存储额外来源信息。

示例：

代码块

```
1  {
2    "source_software": "toppic_topfd",
3    "profile": "topdown_prsm",
4    "capabilities": {
5      "has_ms1": true,
6      "has_ms2": true,
7      "has_envelopes": true,
8      "has_prsms": true,
9      "has_matched_peaks": true,
10     "has_annotated_sequence": true,
11     "has_multi_scan": true
12   }
13 }
```

4.2 ResultSet

4.2.1 现有表名

当前数据库表叫：cutoffs，这来自 TopPIC 语义。

未来在设计上会把它理解为：ResultSet，也就是“一套分析结果视图”。

4.2.2 现有字段

- `id`
- `dataset_id`
- `kind`
- `label`
- `data_path`

4.2.3 必须满足

- `dataset_id` 必须指向存在的 dataset。
- 同一 dataset 下 `kind` 唯一。
- `kind` 当前可为：
 - `prsm`

proteoform

- label 非空。
- data_path 可追踪来源数据位置。

4.2.4 未来扩展

如果未来来源没有 cutoff 概念，也仍然可以创建一个 ResultSet：

代码块

```
1 kind = "default"
2 label = "Default result set"
```

不会因为外部格式没有 cutoff 这个词就无法入库。

4.3 Protein

4.3.1 现有表名

当前数据库表叫：proteins

Protein 表用于保存某个 ResultSet / Cutoff 下的蛋白或序列条目，是 Proteoform 和 PrSM 的上层归属对象。

4.3.2 现有字段

- id
- cutoff_id
- sequence_id
- sequence_name
- sequence_description
- compatible_proteoform_number
- prsm_number
- best_prsm_id
- best_prsm_e_value

其中：

- id 是数据库主键。
- cutoff_id 指向 cutoffs.id，表示该 protein 属于哪个 ResultSet / Cutoff。
- sequence_id 是来源软件中的蛋白或序列业务 ID。

- `sequence_name` 是蛋白或序列名称，当前列表、详情和搜索会使用。
- `sequence_description` 是蛋白描述，例如物种、基因名、数据库描述；可为空。
- `compatible_proteoform_number` 是该 protein 下兼容的 proteoform 数量。
- `prsm_number` 是该 protein 下关联的 PrSM 总数。
- `best_prsm_id` 是该 protein 下最优 PrSM 的业务 id；可为空。
- `best_prsm_e_value` 是最优 PrSM 的 e-value；可为空。

4.3.3 必须满足

如果数据集声明支持 `topdown_prsm` 完整功能，入库时必须保证：

- `cutoff_id` 必须指向存在的 `cutoffs.id`。
- `sequence_id` 必须存在；同一 `cutoff_id` 下 `sequence_id` 唯一。
- `sequence_name` 必须存在；如果外部格式没有名称，Adapter 应生成稳定名称。
- `compatible_proteoform_number` 必须存在；没有统计时可填 `0`，但完整 Top-down 数据建议真实填写。
- `prsm_number` 必须存在；没有统计时可填 `0`，但完整 Profile 建议与子项数量一致。
- 如果填写 `best_prsm_id`，它必须能在同一 cutoff 的 `prsms.prsm_id` 中找到。
- 如果填写 `best_prsm_e_value`，它应与 `best_prsm_id` 对应。

4.3.4 已有的非必须字段

这些字段当前数据库已经存在，不是未来新增字段：

- `sequence_description`
- `best_prsm_id`
- `best_prsm_e_value`

建议填写原因：

- `sequence_description` 有助于搜索、详情展示和人工识别。
- `best_prsm_id` 可用于从 protein 快速跳转到最佳 PrSM。
- `best_prsm_e_value` 可用于列表排序和质量判断。

4.3.5 未来新增字段

为了接收其它格式，未来增加：

- `accession`：蛋白数据库 accession，例如 UniProt accession。可空；若来源能提供，建议写入；应与 `sequence_name` 区分。

- `sequence`：氨基酸完整序列。可空；若来源能提供，建议写入；适合长文本存储。
- `extra_metadata`：JSONB，用于存储来源软件特有的 protein 级字段；Adapter 不应把关键关系字段只塞进这里。

4.3.6 约束

- 同一 `cutoff_id` 下 `sequence_id` 唯一。
- 如果外部格式没有 protein，则不能声明完整 `topdown_prsm` 能力。

4.4 Proteoform

4.4.1 现有表名

当前数据库表叫：proteoforms

Proteoform 表用于保存某个 Protein 下的蛋白异构体，是 PrSM 的直接上层归属对象。

4.4.2 现有字段

- `id`
- `cutoff_id`
- `protein_id`
- `proteoform_id`
- `sequence_id`
- `sequence_name`
- `proteoform_mass`
- `prsm_number`
- `best_prsm_id`
- `best_prsm_e_value`
- `n_acetylation`
- `unexpected_shift_number`

其中：

- `id` 是数据库主键。
- `cutoff_id` 指向 `cutoffs.id`，表示该 proteoform 属于哪个 ResultSet / Cutoff。
- `protein_id` 指向 `proteins.id`，表示该 proteoform 属于哪个 protein。
- `proteoform_id` 是来源软件中的 proteoform 业务 id，不是数据库主键。
- `sequence_id` 是来源软件中的蛋白或序列业务 ID，应与上层 protein 对应。

- `sequence_name` 是该 proteoform 所属序列名称。
- `proteoform_mass` 是 proteoform 质量；可为空。
- `prsm_number` 是该 proteoform 下关联 PrSM 数量。
- `best_prsm_id` 是该 proteoform 下最优 PrSM 的业务 id；可为空。
- `best_prsm_e_value` 是最优 PrSM 的 e-value；可为空。
- `n_acetylation` 是 N 端乙酰化信息；当前 importer 暂时写入 `None`。
- `unexpected_shift_number` 是 unexpected mass shift 数量；当前 importer 暂时写入 `None`。

其中 `proteoform_mass`、`best_prsm_id`、`best_prsm_e_value`、`n_acetylation`、`unexpected_shift_number` 属于当前已有的可选但建议填写字段：

- `proteoform_mass` 有助于质量展示和筛选。
- `best_prsm_id` 可用于从 proteoform 快速跳转到最佳 PrSM。
- `best_prsm_e_value` 可用于列表排序和质量判断。
- `n_acetylation` 和 `unexpected_shift_number` 有助于展示修饰特征；如果来源不能提供，可为空。

4.4.3 必须满足

如果数据集声明支持 `topdown_prsm` 完整功能，入库时必须保证：

- `cutoff_id` 必须指向存在的 `cutoffs.id`。
- `protein_id` 必须指向存在的 `proteins.id`。
- `proteoform_id` 必须存在；它是业务 id。
- `sequence_id` 必须存在，并应与上层 protein 的 `sequence_id` 对应。
- `sequence_name` 必须存在；如果来源没有名称，Adapter 应生成稳定名称。
- `prsm_number` 必须存在；没有统计时可填 `0`，但完整 Profile 建议与子项数量一致。
- 如果填写 `best_prsm_id`，它必须能在同一 cutoff 的 `prsms.prsm_id` 中找到。
- 如果填写 `best_prsm_e_value`，它应与 `best_prsm_id` 对应。

4.4.4 未来新增字段

为了接收其它格式，建议未来增加：

- `modifications`：JSONB，用于保存修饰位点、质量偏移、修饰类型等结构化信息。
- `extra_metadata`：JSONB，用于保存来源软件特有的 proteoform 级字段；Adapter 不应把关键关系字段只塞进这里。

4.4.5 约束

- 同一 `(cutoff_id, protein_id, proteoform_id)` 唯一。
- PrSM 必须能关联到某个 Proteoform。

4.5 PrSM

4.5.1 现有表名

当前数据库表叫: prsms

PrSM 表用于保存 proteoform 与 spectrum 的匹配结果, 是当前详情页、序列注释、匹配峰表和谱图联动的核心数据表。

4.5.2 现有字段

- `id`
- `cutoff_id`
- `proteoform_id`
- `prsm_id`
- `sequence_id`
- `p_value`
- `e_value`
- `fdr`
- `matched_fragment_number`
- `matched_peak_number`
- `spectrum_file_name`
- `ms1_scans`
- `ms2_scans`
- `ms1_ids`
- `ms2_ids`
- `precursor_mono_mass`
- `precursor_charge`
- `precursor_mz`
- `feature_inte`
- `proteoform_mass`

- `ms_header`
- `ms_peaks`
- `annotated_protein`

其中：

- `id` 是数据库主键。
- `cutoff_id` 指向 `cutoffs.id`，表示该 PrSM 属于哪个 ResultSet / Cutoff。
- `proteoform_id` 指向 `proteoforms.id`，表示该 PrSM 属于哪个 proteoform。
- `prsm_id` 是来源软件中的 PrSM 业务 id；详情页 URL 当前按它查询。
- `sequence_id` 是来源软件中的蛋白或序列业务 ID。
- `p_value`、`e_value`、`fdr` 是质量评估指标；可为空。
- `matched_fragment_number`、`matched_peak_number` 是匹配统计；可为空。
- `spectrum_file_name` 是来源谱文件名；可为空。
- `ms1_scans`、`ms2_scans` 是扫描号文本；可为空，但完整图谱联动建议填写。
- `ms1_ids`、`ms2_ids` 是谱图文件定位 id；完整图谱联动必须可解析。
- `precursor_mono_mass`、`precursor_charge`、`precursor_mz`、`feature_inte` 是前体与 feature 信息；可为空，但建议填写。
- `proteoform_mass` 是该 PrSM 对应 proteoform 质量；可为空。
- `ms_header` 是谱头 JSONB。
- `ms_peaks` 是匹配峰 JSONB。
- `annotated_protein` 是序列注释 JSONB。

4.5.3 必须满足

如果数据集声明支持 `topdown_prsm` 完整功能，入库时必须保证：

- `cutoff_id` 必须指向存在的 `cutoffs.id`。
- `proteoform_id` 必须指向存在的 `proteoforms.id`。
- `prsm_id` 必须存在；同一 `cutoff_id` 下必须唯一。
- `sequence_id` 必须存在。
- `ms1_ids` 必须能解析出至少一个 MS1 spectrum id。
- `ms2_ids` 必须能解析出至少一个 MS2 spectrum id。
- `ms_header` 必须保存来源谱头信息。
- `ms_peaks` 必须保存匹配峰信息。

- `annotated_protein` 必须保存序列注释信息。

完整图谱联动还必须满足：

`ms_header` 中必须能提供：

- `ms1_ids`
- `ids` (MS2 ids)
- `ms1_scans`
- `scans` (MS2 scans)

`ms_peaks` 中每个关键 peak 必须能提供：

- `peak_id`
- `monoisotopic_mz`
- `intensity`
- `charge`
- `matched_ions` (完整高亮和标签需要)

`annotated_protein` 中必须能提供：

- `sequence_id`
- `proteoform_id`
- `sequence_name`
- `annotation`

4.5.4 已有的非必须字段

- `p_value`
- `e_value`
- `fdr`
- `matched_fragment_number`
- `matched_peak_number`
- `spectrum_file_name`
- `precursor_mono_mass`
- `precursor_charge`
- `precursor_mz`
- `feature_inte`
- `proteoform_mass`

建议填写原因：

- `p_value`、`e_value`、`fdr` 用于质量评估、排序和筛选。
- `matched_fragment_number`、`matched_peak_number` 用于列表展示和结果质量判断。
- `spectrum_file_name` 有助于追溯来源文件。
- `precursor_mono_mass`、`precursor_charge`、`precursor_mz` 用于前体信息展示和 MS1 标记。
- `feature_inte` 用于 feature 强度展示。
- `proteoform_mass` 有助于详情展示和质量对照。

4.5.5 未来新增字段

为了接收其它格式，建议未来增加：

- `default_ms1_id`：从 `ms1_ids` 中解析出的默认 apex id。
- `default_ms2_id`：从 `ms2_ids` 中解析出的默认 apex id。
- `all_ms1_ids`：JSONB 或数组形式，保存所有可切换 MS1 id。
- `all_ms2_ids`：JSONB 或数组形式，保存所有可切换 MS2 id。
- `extra_metadata`：JSONB，用于保存来源软件特有的 PrSM 级字段。

4.5.6 约束

- 同一 `cutoff_id` 下 `prsm_id` 唯一。
- `PrSM.annotated_protein.sequence_id` 应与 `prsms.sequence_id` 一致。
- `PrSM.annotated_protein.proteoform_id` 应能定位到对应 Proteoform。
- `PrSM.ms2_ids` 必须能定位至少一张 MS2 谱。

4.6 Spectrum

4.6.1 当前存储策略

当前谱图大文件不进入 PostgreSQL。

数据库只保存：

- dataset 路径
- PrSM 中的 ms1/ms2 id
- PrSM 匹配峰信息

谱图文件仍在磁盘上。

4.6.2 当前数据库关联字段

当前没有独立 `spectra` 表。谱图通过以下现有字段间接关联：

- `datasets.source_path`：数据集根路径。
- `prsms.ms1_ids`：MS1 谱图 id。
- `prsms.ms2_ids`：MS2 谱图 id。
- `prsms.ms1_scans`：MS1 scan 文本。
- `prsms.ms2_scans`：MS2 scan 文本。
- `prsms.ms_header`：PrSM 原始谱头 JSONB。
- `prsms.ms_peaks`：PrSM 匹配峰 JSONB。

当前 TopPIC/TopFD 路径规则：

- MS1: `{datasets.source_path}/topfd/ms1_json/spectrum{id}.js`
- MS2: `{datasets.source_path}/topfd/ms2_json/spectrum{id}.js`

4.6.3 必须满足

不管谱图原始格式是什么，应用层最终要能得到统一 Spectrum 结构。

SpectrumV0 必须包含：

- `v`：版本，例如 `"0.1"`。
- `ms`：1 或 2。
- `sid`：谱图 id。
- `rt`：保留时间，单位秒。
- `peaks`：峰数组。

每个 peak 必须包含：

- `m`：m/z。
- `i`：intensity。

4.6.4 完整 Top-down PrSM 能力要求

如果数据集声明支持完整 `topdown_prsm`，MS2 Spectrum 必须包含：

- `envelopes`

每个 envelope 必须包含：

- `id`
- `mm`：monoisotopic mass

- `z` : charge
- `ep` : 同位素子峰数组

并且:

- `envelopes[].id` 必须能与 `PrSM.ms_peaks.peak[].peak_id` 对齐。

4.6.5 多 scan 策略

已确认规则:

- 默认 apex。
- 支持手动切换 all。

因此准入模块应解析并输出:

- `default_sid`
- `all_sids[]`

当前 TopPIC/TopFD 中:

- MS1 来源: `ms.ms_header.ms1_ids`
- MS2 来源: `ms.ms_header.ids`

4.6.6 未来扩展

为了支持更多格式和更稳定的谱图管理, 建议未来增加独立的 `spectra` 或 `spectrum_index` 表。

建议字段:

- `id`
- `dataset_id`
- `ms_level`
- `sid`
- `scan`
- `retention_time`
- `file_path`
- `standard_file_path`
- `peak_count`
- `has_envelopes`
- `extra_metadata` : JSONB

这张表不是第一阶段必须；当前可以继续用 `datasets.source_path + prsms.ms1_ids/ms2_ids` 的方式定位谱图。

5. 数据集能力声明(Capability)

不是所有数据都必须支持完整 PrSM 页面。

因此每个 dataset 应有能力声明。

建议字段：

代码块

```
1  {
2    "has_ms1": true,
3    "has_ms2": true,
4    "has_spectra_peaks": true,
5    "has_envelopes": true,
6    "has_multi_scan": true,
7    "has_proteins": true,
8    "has_proteoforms": true,
9    "has_prsms": true,
10   "has_matched_peaks": true,
11   "has_annotated_sequence": true
12 }
```

5.1 完整当前页面需要

当前完整 PrSM 页面需要：

- `has_ms1`
- `has_ms2`
- `has_spectra_peaks`
- `has_envelopes`
- `has_prsms`
- `has_matched_peaks`
- `has_annotated_sequence`

5.2 如果缺失能力

允许降级：

- 没有 `has_envelopes`：可以画基础 MS2，但不能保证匹配峰局部包络视图。

- 没有 `has_prsms`：只能作为谱图数据集，不进入 PrSM 页面。
- 没有 `has_annotated_sequence`：不能展示完整序列注释。

6. 新增表

6.1 import_runs

6.1.1 原因

多格式导入后，必须能追溯：

- 哪次导入？
- 用哪个 Adapter？
- 成功还是失败？
- 错误是什么？

6.1.2 建议字段

- `id`
- `dataset_id`
- `adapter_name`
- `adapter_version`
- `format_version`
- `started_at`
- `finished_at`
- `status`
- `error_count`
- `warning_count`
- `log_path`
- `summary`：JSONB

6.2 source_files

6.2.1 原因

未来原始文件格式多样，必须登记：

- 原文件在哪里？

- 文件类型是什么?
- 文件有没有被移动或损坏?

6.2.2 建议字段

- `id`
- `dataset_id`
- `path`
- `relative_path`
- `role`
- `file_type`
- `checksum`
- `size_bytes`
- `extra_metadata` : JSONB

6.2.3 role 示例

- `raw_spectrum`
- `standard_spectrum`
- `protein_index`
- `prsm_detail`
- `metadata`

7. 转换模块(Adapter)

7.1 Adapter 是什么

Adapter 是把外部格式变成数据库入库标准的转换器。

每种外部格式至少对应一个 Adapter，例如：

- `toppic_topfd`
- `vendor_x`
- `custom_csv`

7.2 Adapter 输入

- `root`
- `slug`

- name
- description
- format

7.3 Adapter 输出

Adapter 必须输出可校验的标准 payload:

- DatasetPayload
- CutoffPayload[]
- ProteinPayload[]
- ProteoformPayload[]
- PrsmPayload[]
- SpectrumIndexPayload[]
- SourceFilePayload[]
- CapabilityPayload

7.4 Adapter 不能做的事

- 不能绕过校验直接写库。
- 不能把格式差异暴露给前端。
- 不能在缺少关键字段时假装完整支持。

8. 关于入库时间缩短

8.1 原因

入库流程:

代码块

```
1  大量小文件 prsm*.js
2  ↓
3  逐个打开文件
4  ↓
5  逐个解析 JSON
6  ↓
7  逐条/小批量写 PostgreSQL
```

这类流程天然慢, 尤其是 Windows 上大量小文件 IO 会非常明显

目前来说主要是卡在磁盘IO（小文件随机读取+高频打开关闭）和数据库IO（大量写入 PostgreSQL 的往返 IO；目前是已经进行过批次处理，一批进行200条prsm写入）

8.2 解决方法

分为两种模式，一种保持现在的模式全量导入（现在的方案，耗时长查询快），另一种快速上线模式采用轻量登记(耗时短，查询稍慢，如果配置缓存的话二次查询和全量导入速度一致)

轻量登记流程：

代码块

```
1  导入阶段：
2    1. 登记 dataset
3    2. 登记 source_path
4    3. 快速读取 proteins.js 生成列表索引
5    4. 不全量导入所有 prsm*.js 详情
6
7  按需阶段：
8    用户点击某个 PrSM
9      ↓
10   后端读取对应 prsm*.js
11     ↓
12   解析并缓存
13     ↓
14   返回详情
```

这样基本可以控制到30s内完成

两种流程的对比：

全量

```
1  1) 接收导入请求
2    输入：format、root、slug、name、
    description、keep-existing
3
4  2) 基础检查
5    - 检查 root 路径是否存在
6    - 检查必需目录与文件是否存在
    (proteins.js、prsm*.js 等)
7    - 检查数据库连通性
8
9  3) 数据集处理
10   - 按 slug 查 datasets
11   - 若已存在且 keep-
    existing=false：删除旧 dataset（级
    联清理）
```

快速

```
1  1) 接收导入请求
2    输入：format、root、slug、name、
    description、keep-existing、
    mode=fast
3
4  2) 基础检查（只做必要检查）
5    - root 路径存在
6    - 关键目录存在 (topfd、
    proteins.js)
7    - 数据库连通
8
9  3) 数据集登记
10   - 新建或覆盖 datasets (slug、
    name、description、source_path、
    source_software、profile)
```

12 - 新建或复用 dataset (写入 slug、name、description、source_path、source_software/profile 等)

13

14 4) ResultSet/Cutoff 处理

15 - 为每个结果视图创建 cutoff (当前通常 prsm / proteoform)

16 - 记录 kind、label、data_path

17

18 5) 读取 proteins.js (每个 cutoff)

19 - 解析 protein 列表

20 - 计算并写入 proteins

21 - 解析并写入 proteoforms

22 - 维护映射:

23 (sequence_id, proteoform_id) -> proteoforms.id

24

25 6) 全量读取 prsms/prsm*.js (每个 cutoff)

26 - 按文件顺序遍历全部 prsm*.js

27 - 逐文件解析 PrSM

28 - 组装 prsms 行:

29 基础指标 + ms_header + ms_peaks + annotated_protein

30 - 依据映射关联到 proteoform_id

31 - 批量写入数据库 (例如每 200 条一批)

32

33 7) 提交与统计

34 - 事务提交

35 - 输出统计: dataset_id、protein 数、proteoform 数、prsm 数

36 - 写入导入日志 (成功/失败、耗时、错误数)

37

38 8) 导入后验证 (verify_visibility)

39 - 数据集列表可见

40 - cutoff 计数正确

41 - PrSM 列表可查

42 - 抽样校验 ms1_ids/ms2_ids 可定位到谱图文件

11 - 写入 capabilities (先按可识别能力初始化)

12

13 4) ResultSet/Cutoff 快速登记

14 - 创建 prsm / proteoform (或 default) 结果视图记录

15 - 仅登记元信息, 不做深度明细扫描

16

17 5) 快速读取 proteins.js (轻量)

18 - 解析 protein / proteoform 基础信息

19 - 生成列表展示所需索引

20 - 写入 proteins / proteoforms (只写必要字段 + 少量统计)

21

22 6) PrSM 只登记摘要 (不全量导入详情)

23 - 不遍历全部 prsm*.js 详情

24 - 仅登记可快速得到的摘要字段 (prsm_id、e_value 等)

25 - 或先不写 prsms 详情 JSONB (annotated_protein/ms_header/ms_peaks)

26

27 7) 谱图按路径登记, 不导入峰点

28 - 保留 source_path 与文件规则

29 - 记录 default_sid / all_sids (若可快速解析)

30 - 不将全量 spectrum peaks 入库

31

32 8) 提交并上线

33 - 事务提交

34 - 输出快速统计 (dataset/protein/proteoform/摘要 prsm)

35 - 数据集在前端可见并可基础浏览

36

37 9) 按需补全 (Lazy Hydration)

38 - 用户点击某个 PrSM 详情时:

39 读取对应 prsm*.js

40 解析 annotated_protein / ms_header / ms_peaks

41 返回给前端

42 可选: 回写缓存 (内存或数据库)